Fatec Itapira - "Ogari de Castro Pacheco"

Curso Superior de Tecnologia em Desenvolvimento de Software Multiplataforma (DSM)
Disciplina: Computação em Nuvem I
Professor: Prof. Me. Reginaldo Donizeti Cândido

ATIVIDADE PRÁTICA 04

Ecossistema SaaS Multicloud Integrado (Serverless & Frontend)

Objetivo da Atividade

Projetar, desenvolver e implantar um ecossistema SaaS (Software as a Service) composto por três funções independentes (FaaS - Function as a Service) que resolvem um problema de negócio único de forma integrada:

- **Deploy Distribuído:** Realizar a publicação das três funções de forma distribuída entre os três principais provedores de nuvem pública do mercado: **AWS**, **Microsoft Azure** e **Google Cloud Platform (GCP)**.
- **Orquestração Frontend:** Desenvolver e hospedar na nuvem uma aplicação web única (SPA/Dashboard) que orquestre o consumo simultâneo ou encadeado desses microsserviços via requisições HTTP REST.

ATENÇÃO: Esta atividade concentra-se exclusivamente em serviços **PaaS** e **FaaS (Serverless)**, característicos do desenvolvimento moderno de aplicações nativas em nuvem (Cloud-Native), ***eliminando a necessidade de provisionamento e gestão de máquinas virtuais (IaaS) ou contêineres subjacentes***.

1. Orientações Gerais de Conta, Segurança e Boas Práticas

Antes de iniciar o desenvolvimento, o grupo deve observar as seguintes diretrizes técnicas:

1. **Uso de Créditos Educacionais e Free Tier:** Priorizem a utilização dos programas acadêmicos institucionais (*AWS Academy / Educate*, *Azure for Students* e *Google Cloud for Education*) ou as camadas gratuitas (*Free Tier*) de cada provedor.
2. **Padronização de Comunicação (API REST):** Todas as funções devem receber e responder dados estruturados no formato **JSON** (`Content-Type: application/json`), utilizando os códigos de status HTTP apropriados (`200 OK` para sucesso, `400 Bad Request` para validação de parâmetros e `500 Internal Server Error` para falhas internas).
3. **Padrão de Orquestração:** O Frontend pode realizar chamadas **em paralelo** (`Promise.allSettled`) ou **em cascata** (onde a saída de uma função alimenta a entrada da seguinte), desde que o fluxo de negócio seja coeso e transparente para o usuário final.

4. **Segurança e Desativação Pós-Entrega (FinOps):** Como os endpoints estarão públicos para testes em sala, **nunca** exponham chaves de API privadas ou credenciais no código do Frontend ou no GitHub. Após a avaliação em aula, recomenda-se desativar ou excluir os recursos criados para evitar consumo indevido de créditos ou cobranças acidentais.

2. Definição dos Temas e Microsserviços (Por Grupo)

Cada grupo deverá implementar as **três funções** correspondentes ao seu tema. Embora sejam independentes e estejam hospedadas em nuvens distintas, elas devem atuar em conjunto no Frontend para compor a solução SaaS.

Grupo A: SaaS Financeiro (Análise e Projeção de Investimentos)

- **Função 1 (AWS Lambda):** *Calculadora de Juros Compostos* — Recebe valor de aporte inicial/mensal, taxa de juros e tempo (meses), retornando o montante bruto acumulado e o total de rendimentos.
- **Função 2 (Azure Functions):** *Conversor de Câmbio Internacional* — Recebe um valor em Reais (BRL) e simula a conversão do patrimônio projetado para Dólar (USD) e Euro (EUR) com base em cotações pré-definidas ou API pública.
- **Função 3 (Google Cloud Functions):** *Projetor de Inflação e Poder de Compra* — Aplica uma taxa estimada de inflação (IPCA) ao resultado final para calcular o ganho real (descontada a inflação) no período.

Grupo B: SaaS de Recursos Humanos (Gestão de Talentos e Folha)

- **Função 1 (AWS Lambda):** *Analisador de Aderência de Currículo (ATS)* — Analisa um texto de currículo ou lista de competências em busca de palavras-chave da vaga e retorna um *score* de compatibilidade (0 a 100%).
- **Função 2 (Azure Functions):** *Calculadora de Salário Líquido e Encargos* — Recebe a pretensão salarial bruta e retorna o valor líquido estimado após descontos simulados de INSS e IRRF.
- **Função 3 (Google Cloud Functions):** *Recomendador de Trilha de Capacitação* — Com base no *score* e nas lacunas de palavras-chave identificadas, retorna uma lista estruturada de cursos e certificações recomendadas para o candidato.

Grupo C: SaaS de E-commerce (Logística e Checkout Inteligente)

- **Função 1 (AWS Lambda):** *Calculadora de Frete Dinâmico* — Recebe peso do pacote, dimensões básicas e distância/CEP fictício, retornando o custo do frete nas modalidades Econômica e Expressa.
- **Função 2 (Azure Functions):** *Motor de Regras de Cupom e Cashback* — Avalia o valor total do carrinho e a categoria do cliente para gerar um desconto percentual automático e calcular o *cashback* da compra.
- **Função 3 (Google Cloud Functions):** *Estimador de Prazo de Entrega (SLA)* — Retorna a data provável de entrega e o nível de risco logístico com base na região de destino e na modalidade de frete escolhida.

Grupo D: SaaS de Saúde e Bem-Estar (Triagem Nutricional e Performance)

- **Função 1 (AWS Lambda):** *Calculadora de Índices Corporais (IMC e TMB)* — Recebe peso, altura, idade e sexo biológico do paciente, retornando o Índice de Massa Corporal (IMC), a classificação clínica e a Taxa Metabólica Basal (TMB).
- **Função 2 (Azure Functions):** *Planejador de Metas Calóricas e Macronutrientes* — Com base na TMB, no nível de atividade física e no objetivo do usuário (déficit, manutenção ou hipertrofia), retorna a distribuição ideal diária de calorias, proteínas, carboidratos e gorduras.
- **Função 3 (Google Cloud Functions):** *Monitor de Hidratação e Recuperação* — Calcula a meta diária de ingestão de água (em litros) e sugere o tempo recomendado de descanso/sono com base no peso e na intensidade de treino informada.

Grupo E: SaaS de Smart Agro & Sustentabilidade (Monitoramento Agrícola e ESG)

- **Função 1 (AWS Lambda):** *Avaliador de Necessidade Hídrica (Irrigação)* — Recebe dados simulados de sensores (umidade atual do solo, temperatura ambiente e tipo de cultura agrícola) e retorna o volume recomendado de irrigação em litros por metro quadrado (L/m^2).
- **Função 2 (Azure Functions):** *Estimador de Custo Energético e Hídrico* — Recebe o volume de água demandado e o tempo de operação das bombas para calcular o custo financeiro da operação considerando tarifas de horário de ponta e fora de ponta.
- **Função 3 (Google Cloud Functions):** *Calculadora de Pegada de Carbono e Índice ESG* — Avalia o consumo energético da propriedade e a área plantada, retornando a estimativa de emissão de CO_2 e a quantidade de créditos de carbono ou árvores necessárias para compensação ambiental.

3. Circuito de Tarefas

Parte 1: Desenvolvimento e Deploy Distribuído (Backend Serverless)

Cada grupo deve garantir que suas três funções estejam ativas, acessíveis via HTTPS e operando uma em cada provedor:

- **Deploy AWS Lambda:** Exposta publicamente via **Amazon API Gateway** ou **Lambda Function URL**.
- **Deploy Azure Functions:** Configurada com gatilho **HTTP Trigger** (nível de autorização *Anonymous* para fins de teste acadêmico).
- **Deploy Google Cloud Functions (Cloud Run Functions):** Configurada com gatilho **HTTP** permitindo invocações não autenticadas.
- **Configuração de CORS (Cross-Origin Resource Sharing):** Etapa obrigatória em todos os três provedores para permitir que o navegador web acesse os três endpoints a partir do domínio do Frontend sem bloqueios de política de mesma origem (*Same-Origin Policy*).

Parte 2: O Dashboard Integrado (Frontend)

Os alunos devem desenvolver uma aplicação web de página única (HTML5, CSS3/Tailwind/Bootstrap e JavaScript moderno com `fetch` ou `axios`) que será o ponto de união do SaaS:

- A interface deve coletar os dados de entrada do usuário de forma intuitiva e disparar as requisições para as 3 nuvens.

- Deve apresentar os resultados consolidados em um **Dashboard unificado**, identificando visualmente qual provedor processou cada módulo (ex.: selos ou cards indicando *Processed by AWS*, *Azure* e *GCP*, além do tempo de resposta em milissegundos de cada requisição).
- **Desafio de Resiliência (Graceful Degradation / Fallback):** Caso um dos provedores falhe, demore além do tempo limite (*timeout*) ou esteja indisponível, o Frontend **não pode travar**. A interface deve capturar o erro (`try/catch`), exibir um aviso amigável de *"Serviço Temporariamente Indisponível"* apenas naquele módulo específico (ou acionar um cálculo de *fallback* local contingencial) e manter os resultados dos outros dois provedores funcionando normalmente.

Parte 3: Hospedagem da Solução em Nuvem

O Frontend não deve rodar em `localhost` durante a entrega final. Ele deve estar publicado na nuvem utilizando um serviço de **Static Web Hosting** (PaaS/Storage), à escolha do grupo:

- **Opções recomendadas:** AWS S3 (com Static Website Hosting / CloudFront), Azure Static Web Apps, Google Firebase Hosting, Vercel, Netlify ou GitHub Pages.

4. Entregáveis Obrigatórios

A. Documentação Técnica (Arquivo PDF)

Relatório técnico detalhado contendo:

1. **Arquitetura da Solução:** Diagrama visual do ecossistema demonstrando o fluxo completo (Usuário -> Hospedagem Frontend -> Requisições HTTP/JSON -> AWS / Azure / GCP).
2. **Especificação dos Endpoints:** Documentação breve das rotas criadas, exemplos de *payloads* de entrada (Request) e de saída (Response) de cada função.
3. **Análise de Métricas e Desempenho:** Tabela comparativa do tempo de resposta (**Latência** em ms) de cada provedor, avaliando cenários de **Cold Start** (primeira requisição após inatividade) versus **Warm Start** (requisições subsequentes).
4. **Relato Técnico Comparativo:** Análise crítica das facilidades e dificuldades técnicas encontradas pelo grupo nos consoles de gerenciamento de cada nuvem (IAM/Permissões, Configuração de CORS, Logs/Monitoramento e Processo de Deploy).

B. Repositório GitHub

O link do repositório público deve estar organizado seguindo a estrutura de diretórios abaixo:

- `/aws-lambda`: Código-fonte da função AWS e instruções/scripts de configuração.
- `/azure-functions`: Código-fonte da função Azure e arquivos de configuração (`host.json`, `function.json`, etc.).
- `/google-functions`: Código-fonte da função GCP e dependências (`package.json` ou `requirements.txt`).
- `/frontend-app`: Código-fonte completo da interface web unificada.

- `README.md`: Documentação principal do repositório contendo nomes dos integrantes, arquitetura, links ativos do Frontend e dos endpoints, além do passo a passo para reprodução dos testes.

C. Apresentação da Solução em Sala

Demonstração prática (10 a 20 minutos por grupo):

1. **Demonstração "Ao Vivo":** Execução do SaaS funcionando diretamente pela URL pública na nuvem, incluindo um teste prático de resiliência (simulando a queda ou erro de um dos endpoints para provar o funcionamento do *Fallback*).
2. **Auditoria nos Consoles:** Exibição rápida dos painéis da AWS, Azure e GCP comprovando os logs de execução e o tráfego recebido em tempo real.
3. **Defesa Arquitetural e FinOps:** Discussão sobre a experiência de desenvolvimento em cada provedor e estimativa de custos caso o SaaS escalasse para 1 milhão de requisições mensais.

5. Critérios de Avaliação

Critério	Descrição Detalhada	Peso
Integração Multicloud (FaaS)	Sucesso na publicação e comunicação funcional simultânea com as 3 funções distribuídas entre AWS, Azure e GCP via HTTPS.	**40%**
Arquitetura Frontend & Resiliência	Qualidade de usabilidade da interface (UX/UI), hospedagem em nuvem, configuração correta de CORS e tratamento de falhas (*Fallback* / isolamento de erros).	**25%**
Documentação Técnica & Métricas	Clareza do diagrama arquitetural, evidências de implantação, análise comparativa de *Cold/Warm Start* (latência) e relato técnico crítico.	**20%**
Organização do Repositório (GitHub)	Código limpo, padronizado em pastas conforme especificado e documentação completa no arquivo `README.md`.	**15%**

Método de Envio: Upload do arquivo PDF (Documentação Técnica) contendo a URL pública do Frontend e o link do repositório GitHub através do TEAMS até a data limite estipulada em aula.